Professionele Bachelor Toegepaste Informatica

Observability in een hybrid cloud setup met IBM Instana

Rayane Waddah

Promotoren:

Jonas Braem
Johan Cleuren

Integration Designers
Hogeschool PXL Hasselt

Bachelorpaper Academiejaar 2025-2026

Professionele Bachelor Toegepaste Informatica

Observability in een hybrid cloud setup met IBM Instana

Rayane Waddah

Promotoren:

Jonas Braem
Johan Cleuren

Integration Designers
Hogeschool PXL Hasselt

Bachelorpaper Academiejaar 2025-2026

Dankwoord

Abstract

Inhoudsopgave

Contents

Dankwoord ii
Abstract iii
Inhoudsopgave iv
Lijst van gebruikte figuren vi
Lijst van gebruikte tabellen vii
Lijst van gebruikte afkortingen viii
Inleiding 1
I. Stageverslag 2
1 Bedrijfsvoorstelling 2
2 IBM Instana 3
2.1 Overzicht 3
2.2 Observability en monitoring 3
2.2.1 Monitoring 3
2.2.2 Observability 3
2.2.3 Monitoring vs Observability 3
2.3 Architectuur van IBM Instana 4
2.3.1 Instana Agent 4
2.3.2 Instana Backend 4
2.3.3 Data-analyse en verwerking 5
2.3.4 Toegang tot Instana 5
3 Stageopdracht 5
3.1 Omgeving van het project 5
3.2 Gebruik van IBM Instana 5
3.3 Doel van de stageopdracht 5
II. Onderzoekstopic 6
1 Onderzoeksvraag 6
2 Use case 7
2.1 Inleiding 7
2.2 Use Case: Bestellingen en Voorraadbeheer in een Supermarktketen 7
2.3 Motivatie voor de Use Case 7
3 Technische keuzes 8
3.1 Backend 8
3.2 Frontend 8

3.3 ArgoCD 8
Conclusie 9
Reflectie 10
Bronnenlijst 11
Bijlagen 12

Lijst van gebruikte figuren

Lijst van gebruikte tabellen

Lijst van gebruikte afkortingen

IBM: International Business Machines

Inleiding

I. Stageverslag

1 Bedrijfsvoorstelling

Integration Designers maakt deel uit van De Cronos Groep, een groot Belgisch netwerk van technologiebedrijven dat zich richt op IT-consultancy en digitale oplossingen. Binnen dit ecosysteem werkt het bedrijf samen met andere gespecialiseerde organisaties. Daarnaast is Integration Designers verbonden met CornerBriX, een groep bedrijven die zich focust op integratie- en middlewareoplossingen. Door deel uit te maken van deze netwerken kan het bedrijf samenwerken met verschillende partners en gebruikmaken van gedeelde kennis en ervaring.

Integration Designers is een IT-consultancybedrijf dat organisaties helpt om hun digitale systemen beter met elkaar te laten samenwerken. Veel organisaties gebruiken verschillende softwaretoepassingen en databronnen voor hun activiteiten. Wanneer deze systemen niet goed met elkaar verbonden zijn, kan informatie moeilijk gedeeld worden en kunnen processen minder efficiënt verlopen. Integration Designers ondersteunt organisaties bij het opzetten van oplossingen die deze systemen met elkaar verbinden.

Om de werking en prestaties van deze systemen goed te kunnen opvolgen, maakt Integration Designers gebruik van IBM Instana. Deze technologie laat toe om applicaties en systemen te monitoren en inzicht te krijgen in hun prestaties. In het volgende onderdeel wordt IBM Instana verder toegelicht en wordt uitgelegd hoe deze tool gebruikt wordt om de werking van systemen te analyseren en te verbeteren.[1]

2 IBM Instana

2.1 Overzicht

IBM Instana Observability is een observabilityplatform dat helpt bij het analyseren en oplossen van problemen in microservices, gecontaineriseerde applicaties, websites, mobiele applicaties, enzovoort. Instana biedt onder andere automatische monitoring van applicaties, analyse van prestaties, anomaliedetectie en ondersteuning bij het identificeren van de oorzaak van problemen. IBM Instana biedt een volledig inzicht in de gezondheid en prestaties van de applicaties en services. [2]

2.2 Observability en monitoring

Voordat IBM Instana volledig kan worden begrepen, is het belangrijk om eerst het verschil tussen monitoring en observability te begrijpen.[3]

2.2.1 Monitoring

Monitoring verwijst naar het opvolgen van de werking en prestaties van systemen door gegevens zoals metrics en logs te verzamelen en te analyseren. Het doel hiervan is om snel te kunnen zien of een systeem correct functioneert en om mogelijke problemen of storingen te detecteren.

2.2.2 Observability

Observability gaat een stap verder. Naast het verzamelen van gegevens probeert observability ook inzicht te geven in waarom een probleem zich voordoet. Dit gebeurt door verschillende soorten informatie, zoals metrics, logs en traces, met elkaar te combineren. Dit geeft niet enkel aan dat er een probleem is, maar ook welke onderdelen van het systeem daarvoor verantwoordelijk zijn. Op basis van deze gegevens kan ook de gezondheid van een systeem worden beoordeeld en kunnen inzichten worden verkregen om problemen sneller te identificeren en op te lossen.

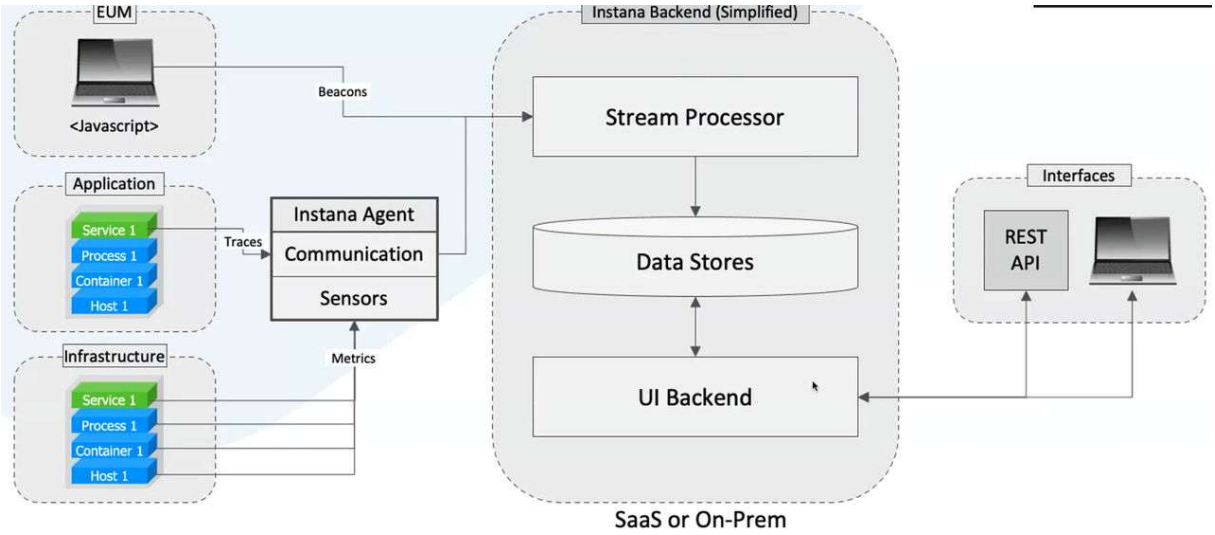
2.2.3 Monitoring vs Observability

Monitoring en observability vullen elkaar aan, waarbij observability extra context biedt om problemen beter te begrijpen en analyseren. Belangrijke soorten gegevens die hierbij gebruikt worden zijn metrics, logs en traces

- Metrics zijn kwantitatieve waardes die gemeten worden over een tijdsperiode. Ze geven een algemeen overzicht van de gezondheid van een systeem, bijvoorbeeld responstijd, CPU-gebruik of netwerkbelasting.
- Logs geven informatie over wat er gebeurd is, wanneer het gebeurd is en waar in het systeem dit plaatsvond.
- Een trace toont het volledige pad dat een verzoek aflegt door verschillende onderdelen van een systeem, bijvoorbeeld wanneer een gebruiker een actie uitvoert in een applicatie. Het is een verzameling van calls. Een call is één stap waarbij een onderdeel van het systeem een ander onderdeel aanspreekt, bijvoorbeeld om gegevens op te vragen of een functie uit te voeren.

2.3 Architectuur van IBM Instana

IBM Instana maakt gebruik van een architectuur die ontworpen is om gegevens in real-time te verzamelen en te analyseren. De architectuur bestaat voornamelijk uit Instana agents, de Instana backend en de gebruikersinterface waarmee de verzamelde informatie kan worden bekeken.[4]



2.3.1 Instana Agent

De Instana Agent is een softwarecomponent die op een host wordt geïnstalleerd. Deze agent monitort automatisch en continu de volledige technologiestack van het systeem. Instana gebruikt een single-agent architectuur, wat betekent dat slechts één agent per host nodig is om gegevens te verzamelen.

De agent detecteert automatisch de technologieën die op het systeem aanwezig zijn en installeert daarvoor specifieke sensoren. Deze sensoren verzamelen gegevens over de prestaties van applicaties en infrastructuur en sturen deze informatie naar de agent. Daarnaast helpen de sensoren bij het verzamelen van trace-gegevens zodat het traject van verzoeken tussen verschillende services kan worden gevolgd.

2.3.2 Instana Backend

De Instana Backend is de centrale component waarin alle verzamelde gegevens worden verwerkt en opgeslagen. Deze backend kan gebruikt worden als een SaaS-oplossing in cloudprovideres zoals AWS of als een self-hosted oplossing op eigen infrastructuur.

Binnen de backend worden verschillende soorten monitoringgegevens verzameld, waaronder:

- Beacons: gegevens die afkomstig zijn van de webbrowser van gebruikers en inzicht geven in de gebruikerservaring van webapplicaties.
- Traces: gegevens die op code-niveau worden verzameld en het traject van verzoeken door verschillende services weergeven.
- Metrics: kwantitatieve tijds gebaseerde gegevens over de prestaties van infrastructuur en systemen.
- Logs: loggegevens die extra context bieden bij gebeurtenissen, foutmeldingen en het gedrag van systemen.

Alle verzamelde gegevens worden met een granulariteit van 1 seconde naar de Instana-backend gestuurd, waar ze worden geanalyseerd en tijdelijk opgeslagen voor verdere verwerking en historische analyse.

2.3.3 Data-analyse en verwerking

Instana verwerkt alle gegevens onmiddellijk nadat ze zijn verzameld. De gegevens worden in real-time gestreamd en geanalyseerd door parallelle verwerkingsprocessen. Hierdoor kan Instana snel inzicht geven in veranderingen binnen een applicatieomgeving, zoals nieuwe code-deployments of veranderingen in infrastructuur.

Door deze continue verwerking kunnen problemen en prestatieveranderingen snel worden gedetecteerd en geanalyseerd.

2.3.4 Toegang tot Instana

Gebruikers kunnen toegang krijgen tot de gegevens die in de Instana backend worden opgeslagen via de Instana webinterface of via de Instana REST API. Via deze interfaces kunnen gebruikers de prestaties van applicaties bekijken, problemen analyseren en inzicht krijgen in de werking van hun IT-omgeving.

3 Stageopdracht

3.1 Omgeving van het project

De stage vindt plaats binnen een omgeving waarin verschillende applicaties en services met elkaar verbonden zijn. Deze systemen draaien deels in de cloud en deels op andere infrastructuur, waardoor er sprake is van een hybrid cloud omgeving.

Een hybrid cloud combineert cloudinfrastructuur met andere IT-omgevingen zodat applicaties en data flexibel beheerd kunnen worden. Binnen deze omgeving wordt gebruikgemaakt van IBM Cloud voor het hosten en beheren van verschillende services en applicaties.

3.2 Gebruik van IBM Instana

Om inzicht te krijgen in de werking en prestaties van de applicaties binnen deze omgeving is gebruikgemaakt van IBM Instana Observability. Instana wordt gebruikt om applicaties en infrastructuur te monitoren en om problemen sneller te detecteren en analyseren.

Door middel van de Instana agents kunnen verschillende onderdelen van de applicatieomgeving automatisch worden gemonitord. De verzamelde gegevens worden vervolgens geanalyseerd in de Instana backend, waardoor inzicht ontstaat in de prestaties van applicaties, services en infrastructuur.

3.3 Doel van de stageopdracht

De stageopdracht richtte zich op het gebruiken en analyseren van IBM Instana binnen de bestaande omgeving. Hierbij is onderzocht hoe Instana kan worden ingezet om beter inzicht te krijgen in de prestaties van applicaties en om mogelijke problemen sneller te identificeren.

II. Onderzoekstopic

1 Onderzoeksvraag

Hoe verhoudt IBM Instana Observability zich tot open-source monitoringtools op vlak van implementatie complexiteit, functionaliteit en beheer bij het realiseren van observability in een hybrid-cloud omgeving?

Deelvragen:

Functionaliteit:

- Hoe verschillen open-source monitoringtools en IBM Instana Observability in de manier waarop telemetry-data wordt verzameld en verwerkt?
- Hoe hanteren beide tools Artificiële Intelligentie in hun oplossing?

Implementatie complexiteit:

- Hoe verhoudt de complexiteit van het toepassen van monitoren op een hybrid cloud omgeving met gebruik van open source tools tegenover IBM Instana Observability?
- Hoe verschillen de kosten tussen open source monitoringtools met de kosten voor IBM Instana Observability in een hybrid-cloud omgeving?

Beheer:

- Hoe verschilt het dagelijks beheer en onderhoud van monitoringinfrastructuur tussen open-source monitoringtools en IBM Instana Observability?
- Hoe verschilt het beheer van monitoringdata (metrics, logs en traces) tussen open-source monitoringtools en IBM Instana Observability?
- Hoe verschilt het beheer van gebruiksrechten tussen open-source monitoringtools en IBM Instana Observability?

2 Use case

2.1 Inleiding

Om de onderzoeksvraag te kunnen beantwoorden wordt een applicatie ontwikkeld op basis van een use case met componenten die zowel on-premise als in de cloud draaien. Op deze manier kan een hybride architectuur gesimuleerd worden waarin verschillende onderdelen met elkaar communiceren.

De use case bestaat uit de volgende componenten:

- Een webinterface die gebruikt wordt als frontend voor de applicatie en waarop End User Monitoring kan worden toegepast (cloud)
- Een applicatie die aanvragen van de webinterface doorstuurt naar een messaging systeem (cloud)
- Een IBM MQ messaging systeem dat communicatie tussen verschillende componenten mogelijk maakt (cloud)
- Een PostgreSQL database waarin gegevens worden opgeslagen (on-premise)
- Een App-Connect-Enterprise-applicatie (ACE) die berichten uit het messaging systeem verwerkt en deze naar de database schrijft (on-premise)

2.2 Use Case: Bestellingen en Voorraadbeheer in een Supermarktketen

Binnen deze studie wordt een fictieve supermarktketen gebruikt als context voor de implementatie van de applicatie. De use case is geïnspireerd op uitdagingen die kunnen voorkomen binnen de bakkerijafdeling van een supermarkt, waar producten vaak een beperkte houdbaarheid hebben en het beheer van de voorraad erg belangrijk is. Hierdoor is een efficiënte verwerking van bestellingen en voorraadbeheer nodig om zowel productbeschikbaarheid als kostencontrole te garanderen.

In de applicatie kunnen gebruikers via een webinterface bestellingen of wijzigingen in de voorraad registreren. Deze aanvragen worden doorgestuurd naar een messaging systeem, dat instaat voor de verdere verdeling van berichten tussen de verschillende onderdelen van de applicatie.

De berichten worden daarna opgehaald door een backend component die de informatie verwerkt en opslaat in een database. Voor deze communicatie wordt gebruikgemaakt van IBM MQ als messaging systeem en een ACE-applicatie voor de verwerking van berichten. De gegevens worden uiteindelijk opgeslagen in een PostgreSQL-database die on-premise draait.

Hierdoor kan met behulp van de observability tools geanalyseerd worden hoe een aanvraag doorheen de verschillende componenten van de applicatie wordt verwerkt.

2.3 Motivatie voor de Use Case

Deze use case werd gekozen omdat het een duidelijke applicatiestroom bevat waarin meerdere systemen met elkaar communiceren. Dit maakt het mogelijk om de verwerking van een request over verschillende componenten heen te analyseren.

Daarnaast maakt dit scenario het mogelijk om verschillende functionaliteiten van de gebruikte technologieën te demonstreren. Door de combinatie van een web interface, messaging systeem, integratie laag en database kan een volledige applicatiestroom opgebouwd worden waarin de observability tools getest en met elkaar vergeleken kunnen worden.

3 Technische keuzes

3.1 Backend

Voor de backend is er gekozen voor Java Springboot. Java ondersteunt automatische instrumentatie via de OpenTelemetry Java Agent. Dit houdt in dat observability-functionaliteiten kunnen worden toegevoegd zonder de code van de applicatie aan te passen. De OpenTelemetry Java Agent wordt toegepast door het online te downloaden en de applicatie op te starten met de volgende opstartoptie [5]:

```
java -javaagent:opentelemetry-javaagent.jar -jar app.jar
```

Hoewel .NET eveneens ondersteuning biedt voor OpenTelemetry, vereist dit in de praktijk vaak dat ontwikkelaars de observability-configuratie expliciet instellen in de code, bijvoorbeeld door ASP.NET- of SQL-tracing handmatig toe te voegen. In tegenstelling hiermee biedt Java een meer plug-and-play benadering via de Java Agent, waardoor automatische telemetrie mogelijk is zonder codewijzigingen voor een groot aantal frameworks en libraries.[6]

3.2 Frontend

Voor de frontend is gekozen voor Vue.js als framework. Vue staat bekend om zijn eenvoudige syntaxis en overzichtelijke component-gebaseerde structuur, waardoor het snel mogelijk is om interactieve gebruikersinterfaces te ontwikkelen en te onderhouden [7]. Deze structuur faciliteert bovendien de integratie van observability-functionaliteiten, zoals het tracen van gebruikersinteracties of API-aanroepen, en sluit daardoor goed aan bij een backend die gebruikmaakt van OpenTelemetry.

Andere populaire frontend-frameworks, zoals React of Angular, zouden eveneens kunnen worden ingezet, maar brengen bepaalde nadelen met zich mee. React vereist vaak extra libraries voor state management of routing, wat de initiële opzet complexer maakt, terwijl Angular een zwaarder framework is met een steilere leercurve, wat voor kleinere tot middelgrote projecten tot onnodige complexiteit kan leiden [8,9]. Vue biedt daarentegen een lichte en flexibele oplossing met een groot ecosysteem van plugins en libraries, wat de ontwikkeling versnelt en het onderhoud vergemakkelijkt [10].

3.3 ArgoCD

Argo CD wordt gebruikt om de kubernetis yaml files te syncen met de manifest in de github repository. Wanneer er in Git iets verandert, wordt Argo CD aangesproken om te synchroniseren en haalt het de nieuwe configuratie op om hem te implementeren. Dit zorgt voor een continue deployment van de applicatie [11].

Conclusie

Reflectie

Bronnenlijst

Voorbeeld:
J. Geralds, "Sega Ends Production of Dreamcast," vnunet.com, para. 2, Jan. 31, 2001. [Online]. Available: http://nl1.vnunet.com/news/1116995. [Accessed: Sept. 12, 2004].

[1] BRON: https://integrationdesigners.com/ 09/03

[2] BRON: https://www.ibm.com/docs/en/instana-observability/1.0.314?topic=overview 09/03

[3] BRON: https://www.ibm.com/think/topics/observability-vs-monitoring 09/03

[4] BRON: https://www.ibm.com/docs/en/instana-observability/1.0.315?topic=overview-architecture 09/03

[5] BRON: https://www.javacodegeeks.com/2026/02/observability-beyond-monitoring-opentelemetry-and-distributed-tracing.html 23/03

[6] https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel 23/03

[7] BRON: https://www.browserstack.com/guide/angular-vs-react-vs-vue?utm 23/03

[8] BRON: https://www.index.dev/skill-vs-skill/frontend-react-vs-vue-vs-angular?utm 23/03

[9] BRON: https://zxweb.eu/en/resources/articles/react-vs-angular-vs-vue-enterprise-comparison 23/03

[10] BRON: https://www.techformation.io/blog/react-vs-angular-vs-vue-js?utm 23/03

[11] BRON: https://argo-cd.readthedocs.io/en/stable/ 25/03

Bijlagen

A. Omschrijving Bijlage A

B. Omschrijving Bijlage B

C. Omschrijving Bijlage C

A. Omschrijving Bijlage A

B. Omschrijving Bijlage B

C. Omschrijving Bijlage C